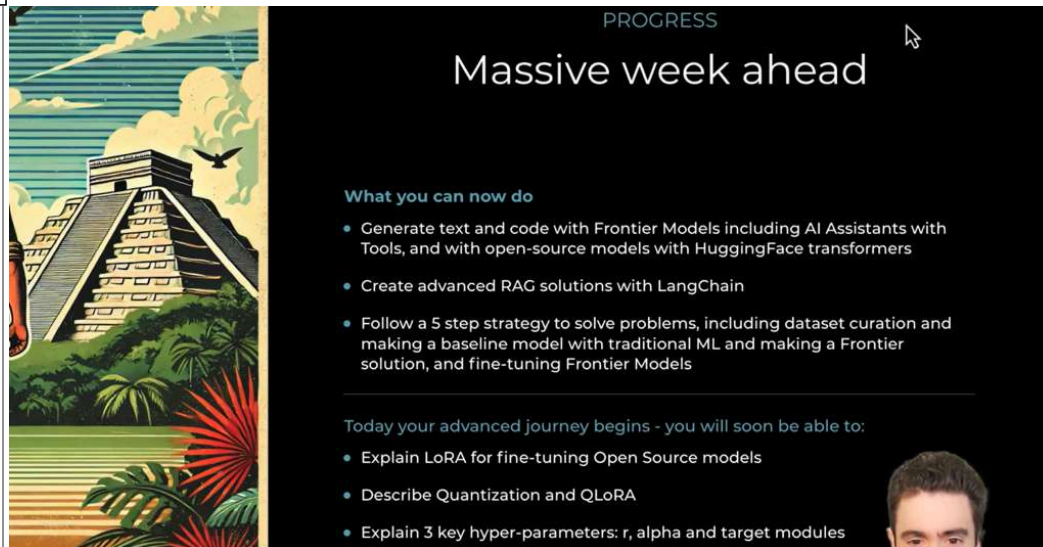


Day 1

02 Tháng Tư 2025 7:12 CH



PROGRESS

Massive week ahead

What you can now do

- Generate text and code with Frontier Models including AI Assistants with Tools, and with open-source models with HuggingFace transformers
- Create advanced RAG solutions with LangChain
- Follow a 5 step strategy to solve problems, including dataset curation and making a baseline model with traditional ML and making a Frontier solution, and fine-tuning Frontier Models

Today your advanced journey begins - you will soon be able to:

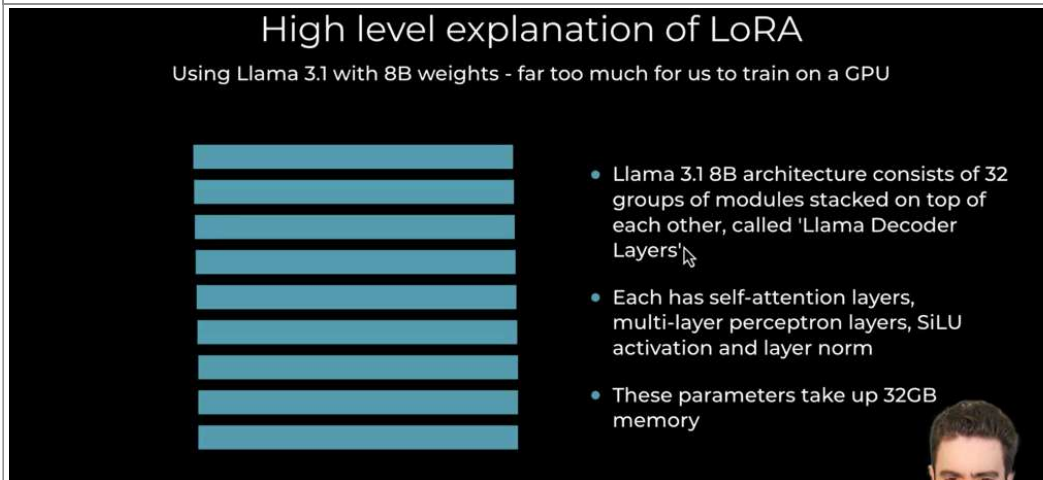
- Explain LoRA for fine-tuning Open Source models
- Describe Quantization and QLoRA
- Explain 3 key hyper-parameters: r , α and target modules

Tuần này sẽ là một tuần quan trọng và nâng cao, giúp học viên đạt được kỹ năng sâu hơn trong lĩnh vực học sâu (deep learning) và các mô hình ngôn ngữ lớn (LLMs). Nội dung chính sẽ tập trung vào:

- Kỹ thuật **LoRA** để tinh chỉnh các mô hình nguồn mở.
- **Lượng tử hóa (Quantization)** và **QLoRA**.
- Ba siêu tham số quan trọng: **R , α và Target Modules**.

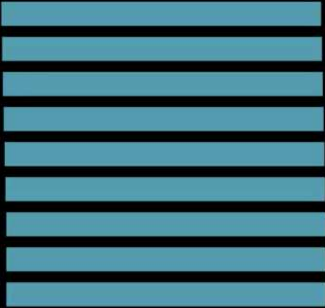
Diễn giả cũng nhắc lại lộ trình tám tuần của khóa học, từ việc sử dụng các giao diện UI, xây dựng pipeline với Hugging Face, đến tuần trước khi mọi người đã thực hành tinh chỉnh mô hình Frontier.

Tuần này, học viên sẽ thực hành tinh chỉnh một mô hình nguồn mở nhỏ hơn, để so sánh hiệu quả với mô hình Frontier.



High level explanation of LoRA

Using Llama 3.1 with 8B weights - far too much for us to train on a GPU



- Llama 3.1 8B architecture consists of 32 groups of modules stacked on top of each other, called 'Llama Decoder Layers'
- Each has self-attention layers, multi-layer perceptron layers, SiLU activation and layer norm
- These parameters take up 32GB memory

Diễn giả bắt đầu phần giới thiệu về **LoRA (Low-Rank Adaptation)**, một kỹ thuật giúp tinh chỉnh mô hình một cách hiệu quả với chi phí thấp. Phần này sẽ có một chút lý thuyết trước khi đi vào thực hành trên **Google Colab**.

Tuần này, học viên sẽ làm việc với mô hình **LLaMA 3.1**, có ba kích thước:

- **8 tỷ tham số**,
- **70 tỷ tham số**,
- **405 tỷ tham số**.

Do hạn chế tài nguyên, ta sẽ sử dụng phiên bản **8 tỷ tham số**, nhưng ngay cả mô hình này cũng yêu cầu **32GB RAM** chỉ để tải vào bộ nhớ, khiến việc tinh chỉnh toàn bộ mô hình trở nên phi thực tế.

Vấn đề tinh chỉnh mô hình lớn:

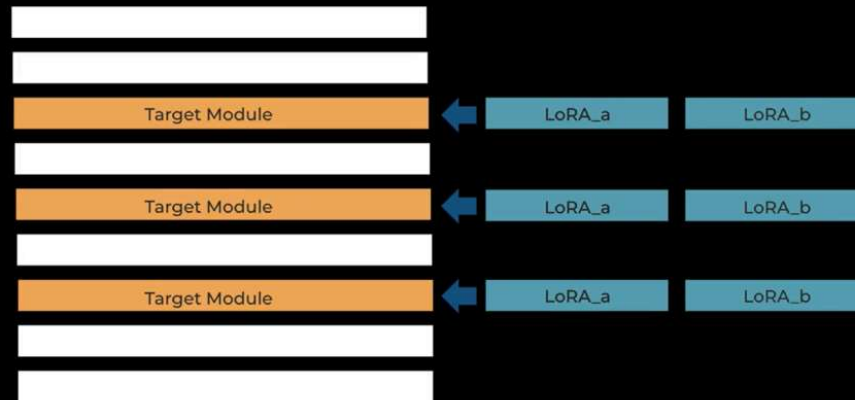
- Việc tối ưu hóa 8 tỷ trọng số tiêu tốn **quá nhiều bộ nhớ và thời gian**.
- Các phòng nghiên cứu lớn như **Meta** có thể chi **hơn 100 triệu USD** để huấn luyện các mô hình lớn.
- Do đó, cần có **kỹ thuật giúp tinh chỉnh mô hình hiệu quả với chi phí thấp**.

Giới thiệu nhanh về kiến trúc LLaMA:

- LLaMA 3.1 có **32 nhóm layer** gọi là **LLaMA decoder layers**.
- Mỗi layer gồm:
 - **Self-attention layers**
 - **Multi-layer perceptron layers**
 - **SiLU activation**
 - **Layer normalization**

High level explanation of LoRA

To be more precise: there are in fact two LoRA matrices that get applied



Diễn giả giới thiệu về **LoRA (Low-Rank Adaptation)** – một kỹ thuật giúp tinh chỉnh mô hình với chi phí thấp. Học viên sẽ thực hành trên **Google Colab** sau một phần lý thuyết ngắn.

Tuần này, khóa học sử dụng **LLaMA 3.1**, với ba kích thước: **8B, 70B, 405B tham số**. Do hạn chế tài nguyên, ta dùng bản **8B**, nhưng ngay cả mô hình này cũng yêu cầu **32GB RAM**, khiến việc huấn luyện toàn bộ trở nên không khả thi.

Vấn đề chính:

- Huấn luyện mô hình lớn tốn **nhiều bộ nhớ, thời gian, chi phí**.
- Các công ty như **Meta** đầu tư **hơn 100 triệu USD** để huấn luyện mô hình lớn.
- Cần một phương pháp tinh chỉnh hiệu quả hơn.

Tổng quan kiến trúc LLaMA 3.1:

- **32 nhóm layer**, mỗi nhóm gọi là **LLaMA decoder layer**.
- Mỗi layer gồm: **Self-attention, MLP, SiLU activation, Layer Norm**.

Diễn giả sẽ trực quan hóa mô hình trên **Colab** để học viên dễ hiểu hơn khi thực hành.

Even the 8B variants are enormous

- **8 Billion * 32 bits = 32GB**
- Intuition: keep the number of weights but reduce their precision
- Model performance is worse, but the impact is surprisingly small
- Reduce to 8 bits, or even to 4 bits

Technical note 1: 4 bits are interpreted as float, not int

Technical note 2: the adapter matrices are still 32 bit

Vấn đề:

- **LLaMA 3.1 (8B)** yêu cầu **32GB RAM**, nhưng GPU T4 chỉ có **15GB** -> Không thể tải cả mô hình vào bộ nhớ.
- **LoRA giúp giảm bộ nhớ nhưng chưa đủ** -> Cần thêm **quantization**.

Giải pháp – Quantization:

- Thay vì giảm số lượng tham số, ta **giữ nguyên số lượng nhưng giảm độ chính xác**.
- Từ **32-bit float** -> **8-bit**, thậm chí **4-bit** (chỉ có 16 giá trị).
- **Hiệu suất vẫn tốt**, chỉ giảm chất lượng một chút.

Kết quả:

- **Giảm đáng kể dung lượng bộ nhớ**, giúp tải mô hình lớn hơn vào GPU rẻ hơn.
- **Base model được quantized, nhưng LoRA adapters vẫn giữ nguyên 32-bit float**.

Diễn giả sẽ tiếp tục giải thích các **hyperparameter quan trọng** trong video tiếp theo.

Three Essential Hyperparameters

For LoRA Fine-Tuning



r

The rank, or how many dimensions in the low-rank matrices

RULE OF THUMB:
Start with 8, then double to 16, then 32, until diminishing returns



Alpha

A scaling factor that multiplies the lower rank matrices

RULE OF THUMB:
Twice the value of r



Target Modules

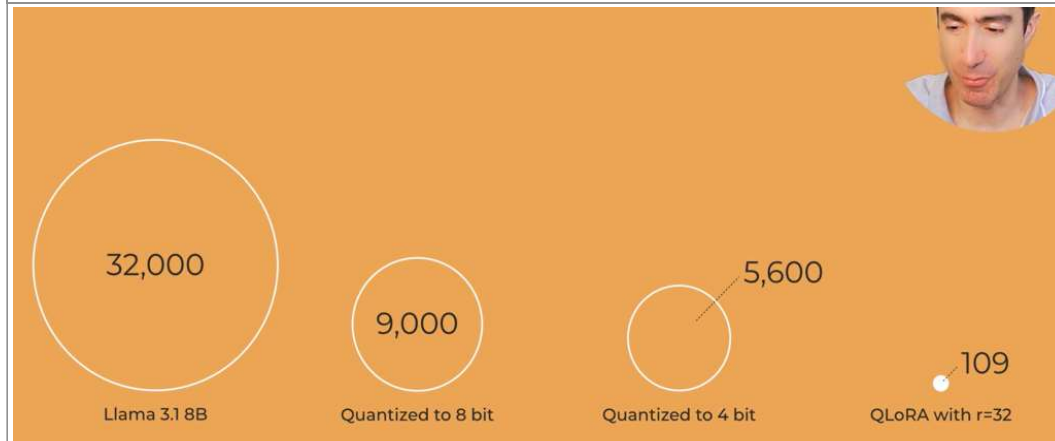
Which layers of the neural network are adapted

RULE OF THUMB:
Target the attention head layers

Bài giảng giới thiệu ba **siêu tham số (hyperparameters)** quan trọng trong quá trình **fine-tuning QLoRA**, bao gồm:

1. **R (Rank - Cấp bậc)**
 - Quyết định số chiều của ma trận hạng thấp trong kiến trúc LLaMA.
 - Giá trị nhỏ giúp tiết kiệm bộ nhớ, chạy nhanh hơn. Giá trị lớn có thể cải thiện hiệu suất nhưng sẽ tốn tài nguyên hơn.
 - Quy tắc thường dùng: bắt đầu với **R = 8**, sau đó thử tăng lên **16, 32,...** đến khi đạt hiệu quả tối ưu.
2. **Alpha (Hệ số tỷ lệ)**
 - Là hệ số nhân ảnh hưởng đến mức độ thay đổi trọng số trong mô hình.
 - Công thức: **Thay đổi trọng số = Alpha × Ma trận A × Ma trận B**.
 - Quy tắc phổ biến: **Alpha = 2 × R** (nếu R = 8, thì Alpha = 16).
3. **Target Modules (Các module mục tiêu cần điều chỉnh)**
 - Chọn các lớp trong mô hình cần được tinh chỉnh.
 - Thường tập trung vào **các lớp attention**, nhưng có thể thay đổi tùy vào mục tiêu bài toán.

Sau phần lý thuyết, bài giảng sẽ chuyển sang **Google Colab** để thực hành với mô hình và quan sát các siêu tham số này trong thực tế.



Bài giảng tổng kết lại lý thuyết về **fine-tuning mô hình Lambda 3.1** và **QLoRA**, đồng thời giới thiệu kế hoạch thực hành tuần tới:

- **Mô hình Lambda 3.1** có 8 tỷ tham số, chiếm **32GB RAM**, nhưng có thể giảm kích thước bằng lượng tử hóa (quantization):
- **8-bit**: còn **9GB RAM**
- **4-bit (double quantization)**: chỉ **5.6GB RAM**
- **Fine-tuning bằng QLoRA** giúp giảm tải bằng cách **huấn luyện các ma trận LoRA riêng biệt**, thay vì huấn luyện toàn bộ mô hình.
- Các ma trận này chỉ có **~109MB**, nhỏ hơn rất nhiều so với mô hình gốc.
- **Kế hoạch tiếp theo:**
- **Chọn mô hình mã nguồn mở** để fine-tuning.
- **So sánh các biến thể** của mô hình.
- **Đánh giá hiệu suất** của mô hình gốc trước khi fine-tuning.
- **Tuần tới sẽ tập trung vào thực hành.**

40

